Permutect: An Elegant Method for Somatic and Germline Variant Calling

David Benjamin[1,4]✉, Ethan Benjamin[3], Lee Lichtenstein[1], Juan Gallegos[2], Sachet Shukla[2], Julian Gascoyne[2], Mehrtash Babadi[1], and Samuel Friedman[1]

[1]The Broad Institute
[2]MD Anderson
[3]Etsy
[4]The author list is not final.

We present Permutect (Permutation-Invariant Mutect), the successor to Mutect2. Permutect combines a novel machine learning model of technical error with a probabilistic model of sample-specific biology, Permutect is able to genotype somatic and germline variants despite being trained using only germline data. By encoding permutation symmetry within a set of reads into its architecture, Permutect can obtain accurate results with very few parameters.

variant calling | somatic | Mutect | deep learning
Correspondence: ***davidben@broadinstitute.org***

Introduction

The first basic idea of Permutect is the simple observation that the set of sequencing reads supporting a variant exhibit permutation symmetry. Genotypes do not depend on which supporting read is considered the first, or seventh etc. Permutect is built from neural networks that respect this symmetry rather than ones that impose an arbitrary order on reads. Doing so permits a model with vastly reduced parameters with no loss of expressive power.

The second idea is to decouple variant calling into two independent parts. Detecting technical errors due to library preparation, sequencing, and alignment is a complex pattern detection problem well-suited to modeling with a neural network. In contrast, distinguishing different types of biological variation is a structured problem that can be modeled explicitly with fewer parameters and varies from sample to sample. Permutect combines a neural network model of technical error with a probabilistic genotyping model. Decoupling of models implies decoupling of training data: the Permutect neural network requires examples of technical error in training but does not require any somatic variants. We call the neural network model of technical error and the probabilistic biology model the artifact model and the posterior model, respectively.

Thanks to these two insights Permutect easily adapts to different sequencing technologies, works for both somatic and germline calling, can be trained on a very modest amount of data, and needs only germline data for training, regardless of the type of calling it needs to do.

In contrast to Permutect's decoupled approach, one could also train a single end-to-end neural network to take a set of reads as input and output a classification as somatic, germline, or non-mutated. Such networks need many examples of somatic variation to train on, which is problematic because labeled somatic truth data are extremely rare, and what little data exist are not always reliable. Furthermore, these truth data are particular to a specific sequencing technology, hence trained models might not generalize to samples that differ from the training samples. Even if sufficient training data were available, end-to-end models require far more parameters than Permutect and are expensive to train and to run.

There is a more pernicious problem: by far the strongest signal distinguishing technical error and somatic mutations from germline variation is the fraction of reads supporting the mutant allele. A fraction near 1/2 or 1 strongly suggests a germline variant, a low fraction is most likely an error, and something in between may be somatic. But what about very pure, monoclonal tumor samples with allele fractions near 1/2? Or cfDNA samples with very low somatic allele fractions? One could include many tumors with a wide variety of cell fractions in the training data, but in addition to complicating things this approach still imposes some arbitrary assumption on what allele fractions are likely. Unlike Permutect, the end-to-end approach needlessly bakes an implicit distribution of somatic allele fractions into the trained model and has no capacity to adapt to the actual distribution found in a particular sample.

In effect, Permutect decomposes variant calling into several questions[1]:

1. Are the mutant allele reads likely to be sequencing errors given the base qualities and the number of variant and non-variant reads?
2. Do the reads support a germline mutation?
3. Are the mutant allele reads a technical or informatic artifact (DNA damage in the laboratory, mapping errors etc)?

Of these, the first two questions are exercises in probability and do not require a machine learning model with many parameters. The third question demands identifying patterns of error that are *not* modeled well by base qualities and an assumption of independent reads. It is difficult and well-suited

[1]Permutect doesn't actually impose these as distinct criteria, rather different parts of the model contribute likelihoods from which posterior probabilities are computed jointly.

DRAFT

to deep learning. However, it has nothing at all to do with identifying somatic variants! Instead of the two classes somatic and non-somatic, we train this deep learning model with data that either artifact or non-artifact. In practice we simply use germline variants as our non-artifact training data.

An attentive reader should object twice to this choice. First, germline variants are not the same as somatic variants. For example, certain substitutions are much more likely in some cancer types and some loci in the genome are hotspots for somatic mutation [TODO: cite COSMIC and maybe a CHIP paper]. This would be a compelling objection for a monolithic end-to-end approach, but in Permutect the prior probabilities of different somatic variants are the responsibility of the posterior model, not the artifact model. As a concrete example, suppose we have an A->T mutation within the context CGGCAGGCC. How likely an A->T mutation is to occur here is the purview of a few-parameter probabilistic model that Permutect fits anew for each sample. The artifact model is only concerned with the question: did a set of reads exhibiting the pattern CGGCTGGCC come from DNA with that same pattern, or did they arise from technical error and a DNA fragment with the CGGCAGGCC allele?

The second objection is that germline variants occur with allele fractions that do not in general represent those found in tumors. To circumvent this, Permutect's artifact model ignores allele fractions by design[2]. Like the prior probability of different mutations, the characteristic allele fractions of somatic variants is a sample-specific property that doesn't require very many parameters. In addition to the fraction-agnostic model architecture we also downsample reads supporting germline variants in order to generate balanced training data where artifacts and non-artifacts occur with a similar range of read counts. That is, after downsampling artifact and non-artifact data are equally likely to have any given number of supporting reads.

Evaluations

Simulated Tumors. Reliable truth data for somatic mutations is extremely rare. One can generate simulated tumors with software, such as BAMSurgeon(1), that inserts substitutions and indels into real sequencing data. The virtues of this approach are first that the set of true mutations is known perfectly as it is specified by the program and secondly that all technical errors in the data are completely real, being the same unmodified errors that were present in the original BAM file. While it is true that the mutations generated in silico are not real, they *are* realistic, since a base changed in silico is indistinguishable from a base changed by mutation and sequenced correctly. The only unrealistic aspect is that the types of mutations – the relative amounts of different substitutions and indel sizes – do not reflect a real tumor. Nonetheless, as long as a good variety of mutations are simulated, data generated in this way present basically the same challenges of error detection as real data.

[2]More precisely, it ignores allele fractions except for calibrating its confidence.

Our simulated data includes the popular DREAM challenge WGS tumor-normal pairs(2), as well as our own synthetic data that we generated from Illumina HiseqX and Novaseq whole genomes sequenced at the Broad Institute. Whereas the DREAM challenge pairs split the reads from a single normal BAM file and add mutations in silico to one of the halves to form a "tumor" and "normal", our simulated tumor and normal come from independently-sequenced replicates of the same normal sample, adding a slight layer of extra realism. To model different tumor purities, we generated synthetic tumors at various somatic mutation allele frequencies.

SEQC2 sample. Because wet lab techniques for validating somatic mutations are very expensive, it is tempting to instead determine truth as the consensus between different variant calling pipelines. In our opinion this usually results in "truth" data of dubious quality. The errors of different algorithms are not independent, and therefore relying on a majority vote among them only gives an illusion of statistical power. For example, variants with low allele fraction are difficult for all algorithms; hence consensus callsets tend to omit such variants.

The SEQC2 Consortium(3) have meticulously created truth data for a breast cancer cell line that avoids the usual shortcomings of consensus data. Rather than simply combining several variant calling algorithms, they used multiple variant callers, multiple alignment algorithms, multiple sequencing technologies, and multiple sequencing centers. Additionally, they validated candidate mutations with extremely deep (greater than 2000x coverage) targeted sequencing. Finally, they validated a random subset of their results with wet lab techniques and showed that their high-confidence truth set is very reliable.

Methods

Labeling training data. One creates a Permutect dataset by running Mutect2[3] in a special Permutect dataset mode. If a germline truth VCF is available, any variant contained in that VCF is considered to be a non-artifact example[4] while any Mutect2 call missing from that VCF is considered an artifact. Otherwise, any variant that is common in the population and is supported by a sufficient fraction of reads is considered a non-artifact[5] and anything called by Mutect2 (without running any subsequent artifact filtering such as FilterMutectCalls) that is supported by a small fraction of reads and is rare in the population is considered an artifact. Everything else is considered unlabeled data and is used in semi-supervised learning.

There are a few important subtleties to this. First, "called by Mutect2" means "called by Mutect2 with sufficient log-odds". The log-odds emitted by Mutect2 is the log likeli-

[3]This is one good reason not to name it Mutect3.

[4]Recall that the Permutect neural network is responsible for classifying artifacts vs non-artifacts, not somatic vs germline variants.

[5]Because germline variants are plentiful these thresholds can be fairly strict without sacrificing too much training data.

DRAFT

hood ratio of a variant versus a sequencing error and is calculated using base qualities with the assumption of independent reads. A sequencing error in Permutect is not the same thing as a technical artifact! We define the latter as an error that is *not* described by base qualities and the assumption of independent reads. That is, a technical artifact depends on some hidden covariate that affects all reads. For example, if there is a single non-reference read with a base quality of 30, the log-odds from Mutect2 will be roughly 3. This is a sequencing error because it comes from nothing more nefarious than the possibility of error as described by the base quality. The base qualities "did their job", so to speak. In contrast, 10 incorrect reads with base qualities of 30, amounting to a log-odds of 30 or so are overwhelmingly unlikely to be due to independent errors each within a probability of 1 in 1000. It is much likelier that there is a common explanation, such as poor mappability, explaining all the reads' errors. We consider such a case to be an artifact. We do not need a neural network to learn how to translate from a phred-scaled base quality to an error probability, nor would we want our training data to drown in such trivial examples. Permutect's neural network artifact model is only concerned with errors that would fool an independent-reads model.

The next subtlety is the balance of training data. For most sequencing technologies, true technical artifacts (as opposed to sequencing error as described above) are much rarer than germline variants. Thus we discard most non-artifact examples, which come from downsampling germline variants, in order to obtain, by default, 10 non-artifacts for every artifact in the training data. When training the artifact model, Permutect counts the data for each label, ref read count, alt read count, and variant type, and balances the loss function automatically. For example, if there are 8 times as many non-artifact substitutions supported by 3 alt reads and 11 ref reads in the training data, then the loss function for artifact substitutions with the same ref and alt read counts is weighted 8 times as strongly.

The final subtlety is the allele fraction criterion when no truth VCF is available to label training data. Marking calls with low (high) non-reference read counts as artifacts (non-artifacts) seems to betray the principles of Permutect by hard-coding the very count dependence we seek to eliminate. This is not the case because Permutect is incapable by design of acquiring such a bias. In the artifact model's gated MLP attention, each read only sees the average of other reads, which erase knowledge of read counts. The scheme of downsampling and dataset balancing adds additional security against Permutect using allele fractions. This weak labeling may introduce some errors into the training data, but as long as technical artifacts *generally* occur with low allele fractions the artifact model will still learn to recognize the correct patterns, though perhaps with less confidence than it should have.

Preparing training data. Next, data must be converted into tensors for computation. We designed this with simplicity in mind and we expect fancier approaches to be fruitful in the future. A single input[6] to the artifact model consists of i) a set of reads supporting the reference allele ("ref"), and ii) a set of reads supporting the non-reference allele ("alt"), with each read encoded as a vector; iii) 21 bases of the reference and alt haplotypes, centered at the variant start position[7]; and iv) a vector of additional features pertaining to the variant. We will now describe each of these in detail,

Reads are encoded as a vector of features: the read mapping quality, the base quality at the variant start, the binary read strand, the binary read pair orientation, the distance of the variant from the left and right ends of the read and of the fragment, the fragment length, and the number of substitution and indel mismatches of the read with respect to its assembled haplotype (not the reference haplotype!) at various distance thresholds from the variant. The reference and alt haplotypes, which are assembled in Mutect2[8], are each represented as five-channel one-hot encodings, with one channel each for A, C, G, T and a fifth deletion channel. An insertion in the alt haplotype relative to the reference is represented as a deletion in the reference allele. For example, for reference sequence ACGTGCA an insertion T -> TT is represented as a reference array ACGTDGC and an alt array ACGTTGC prior to one-hot encoding. (The somatic variant caller NeuSomatic(4) uses the same scheme.) The extra information ("info") vector consists of one element each for the number of STR repeats of repeat length 1, 2, 3, 4, 5, and 6 adjacent to the left and right of the variant, and a few Mutect2 annotations describing the complexity of the local assembly, such as the proportions of reads that align to the three best locally-assembled haplotypes. These features are empirically effective though possibly somewhat redundant; we leave their improvement to future work.

Finally, the non-binary elements of the read and info vectors are quantile normalized. That is, an element in the eg 65th percentile of its corresponding feature is normalized to the point in the unit normal distribution at which the CDF is 0.65. Binary elements such as strand and read orientation are not normalized. After normalization, a single datum contains sets $\{\mathbf{a}_i\}$ and $\{\mathbf{r}_j\}$ of alt and ref reads, a two-dimensional (one dimension for the sequence, one for the channel) tensor $\mathbf{h}$ representing the ref and alt haplotypes, and a vector $\mathbf{e}$ for the extra info.

Artifact Model Overview. The artifact model (see Figure 1) starts with the sets of ref and alt read vectors, the ref and alt haplotypes, and the variant feature vector. It first processes all vectors independently, applying an MLP to all reads, a distinct MLP to the variant feature vector, and a one-dimensional convolutional network to the haplotypes. Then

[6] We ignore batching here for simplicity but in practice, a batch of candidate variants is the actual unit of processing.

[7] 21 bases is large enough to encompass almost any sequence motifs, such as homopolymers, that would predispose the sequencing machine to errors, and small enough for computational efficiency. Centering the window at the variant start allows the model to learn that this is a privileged position. A homopolymer where the variant begins is different from one ten bases away.

[8] Recall that Permutect data are derived from Mutect2, run with the Permutect dataset option enabled.

DRAFT

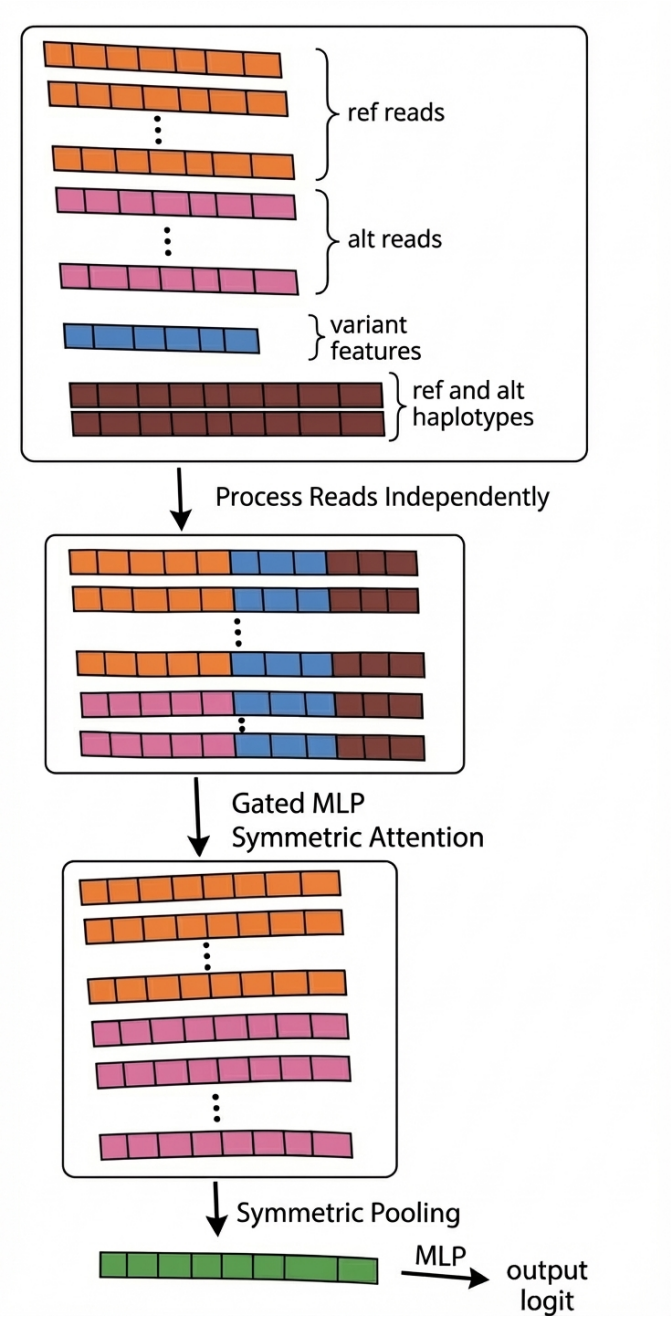

Fig. 1. Artifact model overall view. NOTE: this figure comes from an AI tool to turn hand-drawn sketches into diagrams. IT IS NOT FINAL!!!

the resulting transformed variant feature vectors and haplotype vectors are concatenated with each transformed read vector. At this point, every read vector has been processed in isolation. Next, the processed ref and alt reads are sent through several symmetric gated MLP blocks, which outputs new sets of ref and alt reads, each influenced by attention to other reads. The symmetric gated MLP blocks are both permutation-invariant and count-agnostic. That is, they are invariant to the order of ref and alt reads and each read only pays attention to the average of other reads, which erases knowledge of the total count of ref and alt reads. After the gated MLP blocks, there are still sets of reads that need to be condensed into a single vector. This is accomplished with a symmetric set pooling network, which is also permutation-invariant and count-agnostic. Now the input sets are summarized into a single vector representation of the candidate variant, to which Permutect applies a linear transformation to obtain a single logit expressing the odds that the variant is an artifact.

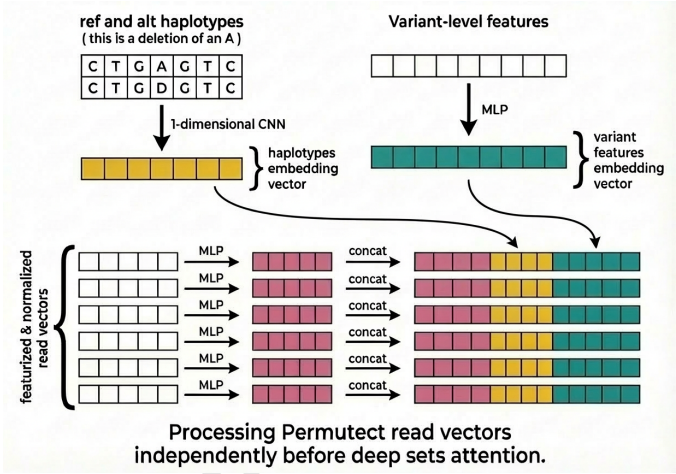

Fig. 2. Processing haplotypes context, variant features, and read vectors before gated MLP inter-read attention.

Artifact Model: bottom layers. Before the artifact model processes the set of reads, it transforms each read vector independently (see Figure 2). In this paper MLP denotes a multi-layer perceptron, a stack of linear transformations separated by a non-linear activation function. We parameterize MLPs by their hidden and output dimensions, eg, $\mathrm{MLP}(10, 20, 10)$ has hidden layers of sizes 10 and 20 and 10-dimensional output. We also allow for residual connections, encoded by negative numbers. For example, suppose we have and MLP $\mathrm{Net} = \mathrm{MLP}(10, -2, 5)$ acting on 7-dimensional input. Then Net contains four linear transformations $L_1 : \mathbb{R}_7 \to \mathbb{R}_{10}$, $L_2 : \mathbb{R}_{10} \to \mathbb{R}_{10}$, $L_3 : \mathbb{R}_{10} \to \mathbb{R}_{10}$, and $L_4 : \mathbb{R}_{10} \to \mathbb{R}_5$ as subnetworks, and the action on an input $\mathrm{x} \in \mathbb{R}_7$ is:

$$\mathbf{x}_1 = \psi(L_1(\mathbf{x})) \tag{1}$$

$$\mathbf{x}_2 = \mathbf{x}_1 + \psi(L_3(\psi(L_2(\mathbf{x}_1)))) \tag{2}$$

$$\mathrm{Net}(\mathrm{x}) = \psi(L_4(\mathbf{x}_2)), \tag{3}$$

where ψ in a nonlinear activation function.

At the bottom of the artifact model, each read vector is transformed independently by an MLP, the info is transformed by a different MLP, and the haplotype data is transformed by a one-dimensional, 10-channel (5 each for ref and alt stacked together) convolutional network. The output of each is a vector:

$$\mathbf{r}_i^{(1)} = \mathrm{MLP}_{\mathrm{ref}}(\mathbf{r}_i) \tag{4}$$

$$\mathbf{a}_i^{(1)} = \mathrm{MLP}_{\mathrm{alt}}(\mathbf{a}_i) \tag{5}$$

$$\mathbf{e}^{(1)} = \mathrm{MLP}_{\mathrm{info}}(\mathbf{e}) \tag{6}$$

$$\mathbf{s}^{(1)} = \mathrm{CNN}_{\mathrm{hap}}(\mathbf{s}). \tag{7}$$

The haplotype and info vectors are then concatenated onto each ref and alt read, independently, to obtain new sets of reads:

$$\mathbf{r}_i^{(2)} = \text{Concat}\left(\mathbf{r}_i^{(1)}, \mathbf{e}^{(1)}, \mathbf{s}^{(1)}\right) \quad \textbf{(8)}$$

$$\mathbf{a}_i^{(2)} = \text{Concat}\left(\mathbf{a}_i^{(1)}, \mathbf{e}^{(1)}, \mathbf{s}^{(1)}\right). \quad \textbf{(9)}$$

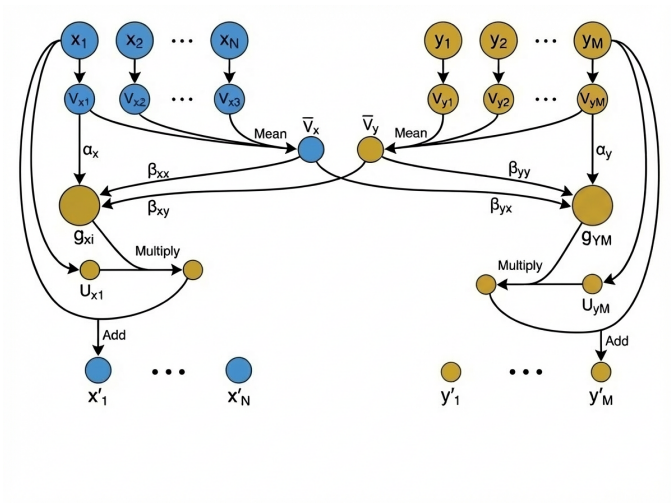

Fig. 3. Symmetric gated MLP block for attention between ref and alt reads. For clarity, only one transformed ref read and one alt read are shown in the output.

Artifact Model: set layers. Next, the transformed reads are passed through a symmetric version of a gated MLP(5) with cross-attention between the alt and ref reads (see Figure 3). We modify the gated MLP to be permutation-invariant and count-agnostic by restricting attention between reads to act on averages. That is, each read is influenced only by the averages of functions of alt and ref reads. Letting LN denote layer norm, Lin denote a single-layer linear transformation, and ψ be some nonlinear activation function, one gated MLP block acts on input sets $\{\mathbf{x}_i\}$ and $\{\mathbf{y}_i\}$ as follows:

$$\mathbf{u}_{x,i} = \psi(\text{Lin}_1(\text{LN}(\mathbf{r}_i))) \quad \textbf{(10)}$$

$$\mathbf{v}_{x,i} = \text{LN}(\psi(\text{Lin}_2(\text{LN}(\mathbf{r}_i)))) \quad \textbf{(11)}$$

$$\mathbf{u}_{y,i} = \psi(\text{Lin}_3^{(n)}(\text{LN}(\mathbf{a}_i^{(n)}))) \quad \textbf{(12)}$$

$$\mathbf{v}_{y,i} = \text{LN}(\psi(\text{Lin}_4(\text{LN}(\mathbf{a}_i)))). \quad \textbf{(13)}$$

The mean fields of two of these transformations are used to make gate vectors, which multiply the other two transformed vectors and are added to the inputs residually to obtain the output:

$$\mathbf{g}_{x,i} = \alpha_x \mathbf{v}_{x,i} + \beta_{xx}\bar{\mathbf{v}}_x + \beta_{xy}\bar{\mathbf{v}}_y \quad \textbf{(14)}$$

$$\mathbf{g}_{y,i} = \alpha_y \mathbf{v}_{y,i} + \beta_{yy}\bar{\mathbf{v}}_y + \beta_{yx}\bar{\mathbf{v}}_x \quad \textbf{(15)}$$

$$\mathbf{x}_i' = \mathbf{x}_i + \mathbf{g}_{x,i} \odot \mathbf{u}_{x,i} \quad \textbf{(16)}$$

$$\mathbf{y}_i' = \mathbf{y}_i + \mathbf{g}_{y,i} \odot \mathbf{u}_{y,i}. \quad \textbf{(17)}$$

where $\bar{\mathbf{v}}_x$ etc denotes the average over all $\bar{\mathbf{v}}_{x,i}$ and $\odot$ is elementwise multiplication. This entire process defines a gated MLP block:

$$\left(\{\mathbf{x}_i'\}, \{\mathbf{y}_i'\}\right) \equiv \text{gMLP}\left(\{\mathbf{x}_i\}, \{\mathbf{y}_i\}\right). \quad \textbf{(18)}$$

The artifact model composes several of these gated MLP blocks to obtain final transformed sets of ref and alt reads:

$$\left(\{\mathbf{r}_i'\}, \{\mathbf{a}_i'\}\right) = \text{gMLP} \circ \cdots \circ \text{gMLP}\left(\left\{\mathbf{r}_i^{(2)}\right\}, \left\{\mathbf{a}_i^{(2)}\right\}\right). \quad \textbf{(19)}$$

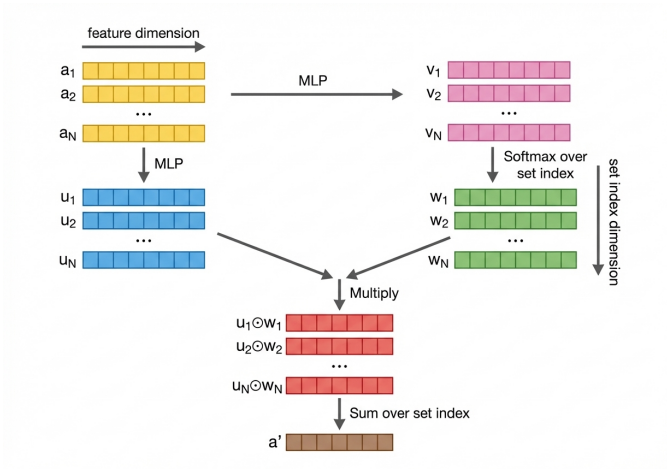

Fig. 4. Pooling a set of transformed reads into a single representation vector for the candidate variant.

At this point we have transformed sets of ref and alt reads in a permutation-invariant manner, but we still need to reduce the set of transformed reads into a scalar. Permutect achieves this with a generative likelihood model that assumes reads are conditionally i.i.d. given a discrete variable indicating different varieties of artifacts. Each potential variant has a one-hot indicator variable $\mathbf{z}$ such that $z_0 = 1$ for non-artifacts and $z_k = 1$ for artifacts of type $k = 1, 2 \ldots K$. If $z_0 = 1$, alt reads are assumed to follow an origin-centered normal distribution:

$$\mathbf{a} \sim N(\mathbf{0}, \mathbf{\Sigma}), \quad \textbf{(20)}$$

where $\mathbf{\Sigma}$ is a learned parameter. Each artifact type k is associated with a unit vector $\hat{\mathbf{v}}_k$ along which reads deviate from the origin. Reads' projections along this direction are assumed to follow exponentially-modified Gaussian (EMG) distributions, while their components orthogonal to this direction are assumed to follow isotropic Gaussian distributions. That is, if $z_k = 1$ for $k > 0$ we have:

$$\mathbf{a} \cdot \hat{\mathbf{v}}_k \sim \text{EMG}(\mu_k, \sigma_k, \lambda_k) \quad \textbf{(21)}$$

$$|\mathbf{a} - \mathbf{a} \cdot \hat{\mathbf{v}}_k| \sim N(0, \tau_k) \quad \textbf{(22)}$$

According to conditional independence, we sum the log-likelihoods of all alt reads given by Equations 20, 21, and 22 to obtain the likelihoods for the indicator $\mathbf{z}$, hence the likelihood and the binary logit for the call of artifact versus non-artifact. In addition to conveniently and efficiently reducing the set of reads to a single binary logit for the variant as a whole, this generative approach automatically encodes a correct probabilistic treatment of different quantities of evidence and numbers of supporting reads. Permutect is more confident of predictions based on more reads. This is worth stressing: a trained Permutect model is not only accurate, its predictions are well-calibrated and its degree of confidence can be trusted.

DRAFT

Training the artifact model

For labeled data the artifact model is trained with the usual cross-entropy loss function. Unlabeled data are given an entropy-minimization loss function that promotes clustering:

$$\text{entropy}(\text{logit}) = -p\log(p) - (1-p)\log(1-p), \quad \textbf{(23)}$$

where $p = \sigma(\text{logit})$ is the mapping from logits to probabilities via the sigmoid function.

The cross entropy loss is minimized when the output probabilities of the model match the actual artifact probabilities given the covariates. Since we train on balanced data containing equal amounts of artifacts and non-artifacts, our training data has balanced prior probabilities, and therefore the learned probabilities that a variant is an artifact given the input data is proportional to the likelihood of the input data given that the variant is an artifact. This observation, that a model trained on balanced data learns not just probabilities but likelihoods, enables the posterior model to use the output of the artifact model, as we shall see below.

Posterior Model. To be precise, the artifact model outputs a conditional log-likelihood ratio given the number of alt and ref reads:

$$\log \frac{\text{P}(\text{reads}|\text{artifact}, n_a, n_r)}{\text{P}(\text{reads}|\text{non artifact}, n_a, n_r)} \quad \textbf{(24)}$$

The end goal, however, is to compute posterior probabilities not only of a candidate variant being somatic or an artifact, but also of being a germline variant or a sequencing error[9]. In order to get posterior probabilities of these four categories of posterior calls we must have: 1) log likelihoods as in Eq. 24 for the two remaining categories of sequencing error and germline variant, 2) prior probabilities for each category, and 3) allele fraction spectra $P(n_a, n_r|\text{call})$ for each call. Given these ingredients, posterior probabilities are obtained by normalizing:

$$P(\text{call}|\text{reads}) \propto P(\text{call})P(n_a, n_r|\text{call})P(\text{reads}|\text{call}, n_a, n_r). \quad \textbf{(25)}$$

At this point, we must confess something that the reader may have suspected earlier. The elegant decoupling between the technology-dependent, machine learning-based artifact model, and the sample-specific, probabilistic posterior model only works because of precise definitions, especially where the two parts come together. We could sweep all this pedantry under the rug with a single black-box model, but instead we persist in our semantic labor. The quantity $P(\text{reads}|\text{artifact}, n_a, n_r)$ means: "given that there exists an artifact appearing in n_a reads, with n_r reads matching the reference allele, what is the probability of observing the featurized read vectors that constitute the input to Permutect's artifact model? Given this, it is clear that $P(\text{reads}|\text{call}, n_a, n_r)$ is the same whether the call is a somatic or germline variant, because in both cases the reads' features are nonanomalous.

[9]Recall the distinction made above between sequencing errors, which occur independently with an error rate given by base qualities, and artifacts.

The priors $P(\text{call})$ for somatic and artifact calls are learned parameters of the posterior model, with one prior probability each for the variant types substitution, single-base deletion, single-base insertion, multi-base deletion, and multi-base insertion. Germline priors are determined by the population allele frequencies if the variant allele is present in a germline resource such as gnomAD [TODO: cite gnomAD 4], or otherwise by a small imputed germline prior. The prior probability of a sequencing error is essentially 1, since the generative process of sequencing error is always present.

We need to model the spectrum $P(n_a, n_r|\text{call})$ for different call types. We model the spectrum of artifacts for each aforementioned variant type as a beta binomial, the shapes of which are learned parameters of the posterior model. Permutect can accept the copy number segmentation from the Mutect2 pipeline (the tool is CalculateContamination) as one of its inputs. In this case, Permutect knows the local minor allele fraction f of germline hets and averages two binomials, with alt probabilities f and $1-f$, as the germline spectrum. Otherwise it uses a minor allele fraction of 1/2, assuming a diploid sample without copy number variation.

The somatic spectrum is a single distribution, not a distinct distribution for each variant type, because it depends only on (sub)clonal cell fractions and copy numbers. It is impossible to design a perfect somatic allele fraction spectrum without solving the entire problem of tumor heterogeneity. However, an effective model for the spectrum that accounts for both subclonality and copy number variation is

$$\theta \sim \sum_k w_k \text{Uniform}(c_k f, c_k(1-f)) \quad \textbf{(26)}$$

$$n_a \sim \text{Binom}(\theta, n), \quad \textbf{(27)}$$

where f is again the minor allele fraction. In the absence of copy number variation the uniform distributions collapse to delta functions and we simply have a binomial mixture of different cell fractions c_k with weights w_k. The uniform distribution when minor allele fractions are not 1/2 acts to smear out somatic allele fractions when there is copy number variation. To understand the rationale behind the form chosen, consider a locus with local copy number 5. Somatic variants could occur on any number of the copies depending on the relative timing of the copy number amplification and the somatic mutation. Excluding the possibility of a loss of heterogeneity deletion, followed by a homozygous somatic mutation on the remaining copy, followed by an amplification, which would yield a somatic mutation of all five copies, we allow for somatic copy numbers of 1, 2, 3, or 4, which we vaguely model as a uniform distribution between 1/5 and 4/5.

The adjustable parameters of the posterior model, the priors and spectra shapes for somatic variants and artifacts are learned via stochastic gradient descent to maximize the sum of log likelihoods given by Eq. 25.

Finally, we cannot easily compute $P(n_a, n_r|\text{call})$ and $P(\text{reads}|\text{call}, n_a, n_r)$ for sequencing error calls. However, their product, or rather the ratio of their product to the corre-

DRAFT

sponding product for somatic calls:

$$\log \frac{P(n_a, n_r|\text{somatic})P(\text{reads}|\text{somatic}, n_a, n_r)}{P(n_a, n_r|\text{seq error})P(\text{reads}|\text{seq error}, n_a, n_r)} \quad \textbf{(28)}$$

is essentially the tumor log odds (TLOD) ouput of Mutect2. Thus we have all the ingredients to compute posterior probabilities for somatic, germline, sequencing error, and artifact calls, allowing joint somatic and germline genotyping.

ACKNOWLEDGEMENTS

We acknowledge Giles Hall for suggesting the use of memory-mapped datasets.

Bibliography

1. Adam Ewing et al. Combining tumor genome simulation with crowdsourcing to benchmark somatic single-nucleotide-variant detection. *Nature Methods*, 12, 2015.
2. Paul Boutros et al. Global optimization of somatic variant identification in cancer genomes with a global community challenge. *Nature Genetics*, 46, 2014.
3. Zhu B. Zhao Y. et al Fang, L.T. Establishing community reference samples, data and call sets for benchmarking cancer mutation detection using whole-genome sequencing. *Nature Biotechnology*, 39, 2021.
4. Sayed Mohammad Ebrahim Sahraeian et al. Deep convolutional neural networks for accurate somatic mutation detection. *Nature Communications*, 10, 2019.
5. Hanxiao Liu, Zihang Dai, David R. So, and Quoc V. Le. Pay attention to mlps. *CoRR*, abs/2105.08050, 2021.

Supplementary Note 1: Something about something

DRAFT